МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 8
з дисципліни «Алгоритмізація та програмування»
на тему «"Реалізація алгоритмів сортування та робота з файлами на мові С ++"»

ХАІ.301.G7.319.30 ЛР

Виконав студент гр._____319гр._____

16.03.2026 Руденко Валентин
(підпис, дата) (П.І.Б)

Перевірив
_____ к.т.н., доц. Олена ГАВРИЛЕНКО
(підпис, дата) (П.І.Б.)

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_*n*.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_*n*.txt».

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_*n*.txt» з елементами вихідного двовимірного масиву (*n - номер варіанта*). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

main.cpp //основна функція і три функції для 3х завдань

array_utils.h //заголовки функцій для роботи з
//одновимірними масивами (завдання1,3)

array_utils.cpp //визначення функцій для роботи з
//одновимірними масивами (завдання1,3)

matrix_utils.h //заголовки функцій для роботи з
//двовимірними масивами (завдання2)

matrix_utils.cpp //визначення функцій для роботи з
//двовимірними масивами (завдання2)

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

ВИКОНАННЯ РОБОТИ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_*n*.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_*n*.txt».

```
void task1_array(const string filename) {
   ifstream fin(filename);
   ofstream fout("array_out.txt");

   int n;
   fin >> n;

   double arr[MAX_SIZE];

   for (int i = 0; i < n; i++)
      fin >> arr[i];

   fout << "Even elements in reverse: ";
   int countEven = 0;

   for (int i = n - 1; i >= 0; i--) {
      if (int(arr[i]) % 2 == 0) {
         fout << arr[i] << " ";
         countEven++;
      }
  }
```

Лістинг коду вирішення задачі наведено в дод. А (стор. 7).
Екрани роботи програми показано на рис. Б.(стор. 10).

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_*n*.txt» з елементами вихідного двовимірного масиву (*n* - *номер варіанта*). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

```
void task2_matrix(const string filename) {
   ifstream fin(filename);
   ofstream fout("matrix_out.txt");

   int m, n;
   fin >> m >> n;

   double matrix[MAX_M][MAX_N];

   for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
         fin >> matrix[i][j];
```

```
    int halfM = m / 2, halfN = n / 2;

    for (int i = 0; i < halfM; i++)
        for (int j = 0; j < halfN; j++)
            swap(matrix[i][j], matrix[i + halfM][j + halfN]);

    fout << "Matrix after swap:\n";

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            fout << matrix[i][j] << "\t";
        fout << "\n";
    }
```

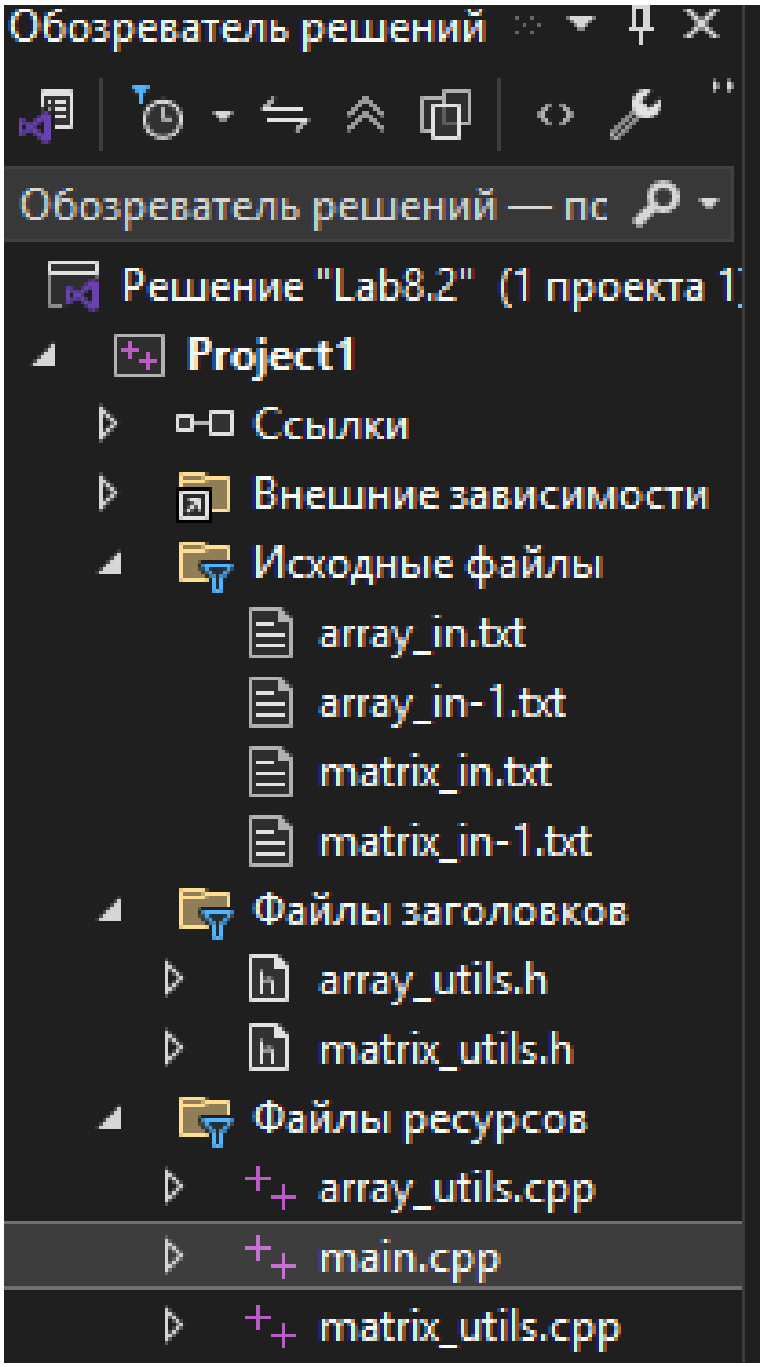
Лістинг коду вирішення задачі наведено в дод. А (стор. 9).
Екрани роботи програми показано на рис. Б.(стор. 11).

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

```
void task3_bubble_sort(const string filename) {
    ifstream fin(filename);
    ofstream fout("array_out.txt");

    int n;
    fin >> n;

    double arr[MAX_SIZE];

    for (int i = 0; i < n; i++)
        fin >> arr[i];

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }

    fout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        fout << arr[i] << " ";
```

Лістинг коду вирішення задачі наведено в дод. А (стор. 8).
Екрани роботи програми показано в дод. Б.(стор. 12).
Діаграма активності для алгоритму вирішення задачі в дод. В (стор. 13, 14)
Схема сортування алгоритму «бульбашки» в дод. Г (стор. 15)

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами.



ВИСНОВКИ

Було вивчено теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізовано оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ДОДАТОК А
Лістинг коду програми

```
#ifndef ARRAY_UTILS_H
#define ARRAY_UTILS_H

#include <string>

void task1_array(const std::string filename);
void task3_bubble_sort(const std::string filename);

#endif



#include "array_utils.h"
#include <iostream>
#include <fstream>
#include <algorithm>

using namespace std;

const int MAX_SIZE = 1000;

// Завдання 1
void task1_array(const string filename) {
    ifstream fin(filename);
    ofstream fout("array_out.txt");

    int n;
    fin >> n;

    double arr[MAX_SIZE];

    for (int i = 0; i < n; i++)
        fin >> arr[i];

    fout << "Even elements in reverse: ";
    int countEven = 0;

    for (int i = n - 1; i >= 0; i--) {
        if (int(arr[i]) % 2 == 0) {
            fout << arr[i] << " ";
            countEven++;
        }
    }
```

```
    fout << "\nNumber of even elements: " << countEven << "\n";

    cout << "Task 1 completed. Check array_out.txt\n";

    fin.close();
    fout.close();
}

// Завдання 3
void task3_bubble_sort(const string filename) {
    ifstream fin(filename);
    ofstream fout("array_out.txt");

    int n;
    fin >> n;

    double arr[MAX_SIZE];

    for (int i = 0; i < n; i++)
        fin >> arr[i];

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }

    fout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        fout << arr[i] << " ";

    fout << "\n";

    cout << "Task 3 completed. Check array_out.txt\n";

    fin.close();
    fout.close();
}


#ifndef MATRIX_UTILS_H
#define MATRIX_UTILS_H

#include <string>

void task2_matrix(const std::string filename);

#endif

#include "matrix_utils.h"
```

```
#include <iostream>
#include <fstream>
#include <algorithm>

using namespace std;

const int MAX_M = 100;
const int MAX_N = 100;

void task2_matrix(const string filename) {
    ifstream fin(filename);
    ofstream fout("matrix_out.txt");

    int m, n;
    fin >> m >> n;

    double matrix[MAX_M][MAX_N];

    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            fin >> matrix[i][j];

    int halfM = m / 2, halfN = n / 2;

    for (int i = 0; i < halfM; i++)
        for (int j = 0; j < halfN; j++)
            swap(matrix[i][j], matrix[i + halfM][j + halfN]);

    fout << "Matrix after swap:\n";

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            fout << matrix[i][j] << "\t";
        fout << "\n";
    }

    cout << "Task 2 completed. Check matrix_out.txt\n";

    fin.close();
    fout.close();
}
```

```
#include <iostream>
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

int main() {
    int choice;
    string filename;

    do {
        cout << "\n===== MENU =====\n";
        cout << "1 - Array analysis\n";
        cout << "2 - Matrix processing\n";
        cout << "3 - Bubble sort array\n";
        cout << "0 - Exit\n";
        cout << "Choose task: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter input file for array: ";
            cin >> filename;
            task1_array(filename);
            break;
        case 2:
            cout << "Enter input file for matrix: ";
            cin >> filename;
            task2_matrix(filename);
            break;
        case 3:
            cout << "Enter input file for array: ";
            cin >> filename;
            task3_bubble_sort(filename);
            break;
        case 0:
            cout << "Bye!\n";
            break;
        default:
            cout << "Wrong choice\n";
        }

    } while (choice != 0);

    return 0;
}
```

ДОДАТОК Б
Скрін-шоти вікна виконання програми

```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 1
Enter input file for array: array_in.txt
Task 1 completed. Check array_out.txt
```

```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 1
Enter input file for array: array_in-1.txt
Task 1 completed. Check array_out.txt
```

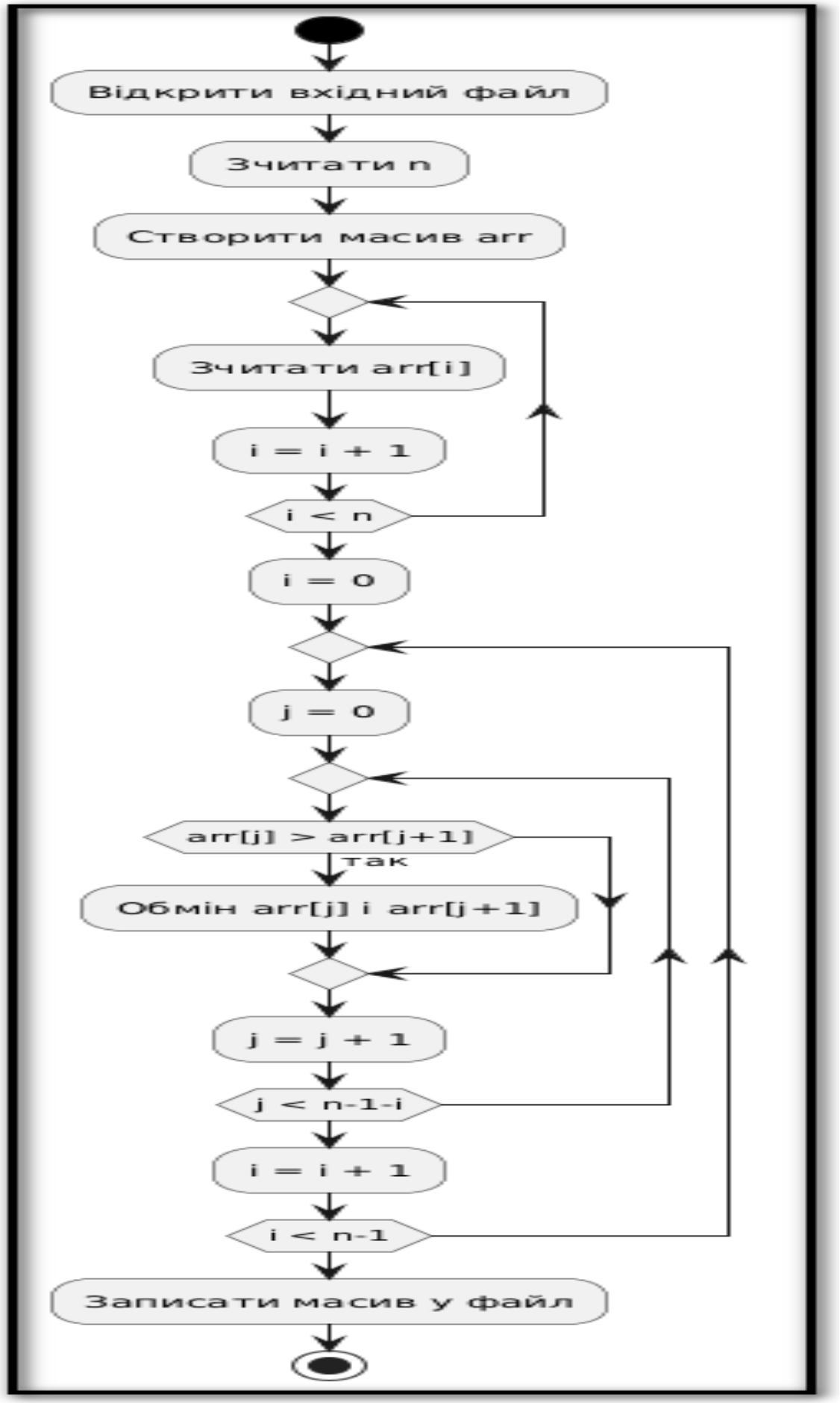
```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 2
Enter input file for matrix: matrix_in.txt
Task 2 completed. Check matrix_out.txt
```

```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 2
Enter input file for matrix: matrix_in-1.txt
Task 2 completed. Check matrix_out.txt
```

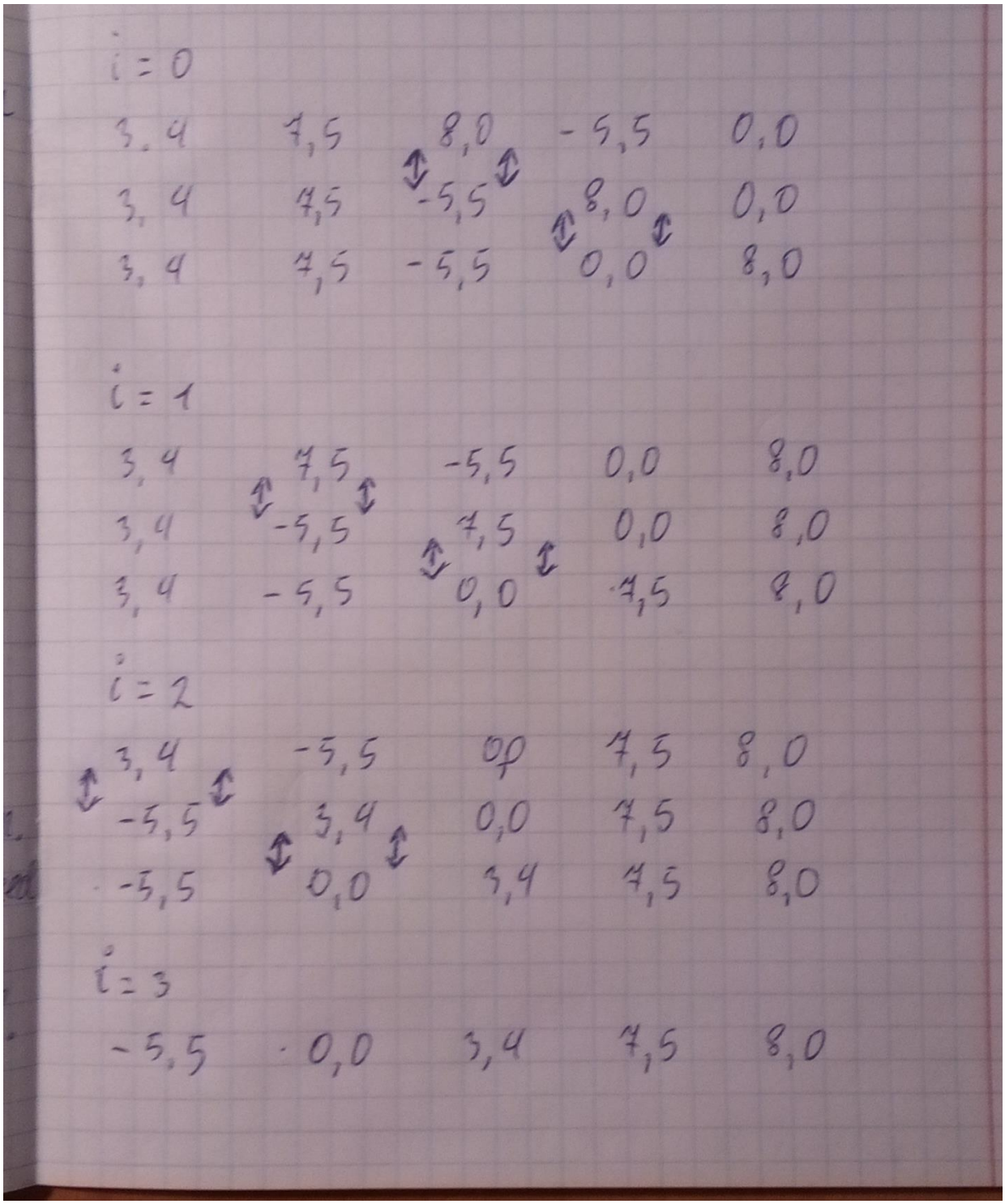
```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 3
Enter input file for array: array_out.txt
Task 3 completed. Check array_out.txt
```

ДОДАТОК В

Відкрити вхідний файл
Зчитати n
Створити масив arr
Зчитати arr[i]
i = i + 1
i < n


Відкрити вхідний файл
Зчитати n
Створити масив arr
Зчитати arr[i]
i = i + 1
i < n
i = 0
j = 0
arr[j] > arr[j+1]
так
Обмін arr[j] і arr[j+1]
j = j + 1
j < n-1-i
i = i + 1
i < n-1
Записати масив у файл

ДОДАТОК Г



i = 0

3,4 7,5 8,0 -5,5 0,0

3,4 7,5 -5,5 8,0 0,0

3,4 7,5 -5,5 0,0 8,0

i = 1

3,4 7,5 -5,5 0,0 8,0

3,4 -5,5 7,5 0,0 8,0

3,4 -5,5 0,0 7,5 8,0

i = 2

3,4 -5,5 0,0 7,5 8,0

-5,5 3,4 0,0 7,5 8,0

-5,5 0,0 3,4 7,5 8,0

i = 3

-5,5 0,0 3,4 7,5 8,0

У процесі сортування стрілками показані елементи, які порівнюються та міняються місцями. На кожному проході найбільший елемент переміщується в кінець масиву. Після кількох ітерацій масив стає повністю відсортованим.

Діалог з ШІ для самоаналізу

Тепер ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4ма варіантами відповіді і три відкритих питання за кодом і теоретичними відомостями у файлах лекції.
+звіт
+текст лекції

1. Тестові питання

1. Яка структура використовується для статичного масиву у коді?
A) vector<double>
B) list<int>
C) int arr[100]
D) array<int, 10>

Правильна: C)

2. Що робить функція swap(arr[j], arr[j+1])?
A) Копіює значення
B) Видаляє елемент
C) Міняє елементи місцями
D) Сортує масив

Правильна C)

3. Яка складність алгоритму бульбашкового сортування?
A) O(n)
B) O(log n)
C) $O(n^2)$
D) O(n log n)

Правильна C)

4. Для чого використовується ifstream?
A) Запис у файл
B) Зчитування з файлу
C) Видалення файлу
D) Робота з пам’яттю

Правильна B)

5. Що відбудеться, якщо файл не відкриється?
A) Програма завершиться автоматично
B) Дані будуть нульові
C) Може виникнути помилка читання
D) Масив автоматично заповниться

Правильна C)

2. Відкриті питання:

1. Поясни принцип роботи сортування бульбашкою

Сортування бульбашкою працює так: ми кілька разів проходимо по масиву і порівнюємо сусідні елементи. Якщо вони стоять не в правильному порядку — міняємо їх місцями за допомогою swap(arr[i], arr[i+1]). Після кожного проходу найбільші елементи поступово "спливають" в кінець масиву.

2. Чому використовується вкладений цикл у сортуванні?

Один прохід по масиву не забезпечує повного сортування. Зовнішній цикл відповідає за кількість проходів по масиву. Внутрішній цикл порівнює сусідні елементи та міняє їх місцями, якщо потрібно. Розмір масиву відомий заздалегідь (n).

3. Які помилки можуть виникнути при зчитуванні файлу?

Файл може не відкритися (неправильне ім'я або шлях). Дані у файлі можуть бути неправильні або менше**,** ніж очікується (n елементів). У цьому випадку програма може працювати некоректно або завершитись з помилкою.

4. Чим статичний масив відрізняється від динамічного?

- статичний масив: розмір задається на етапі компіляції, пам'ять виділяється на стеку, не можна змінити розмір під час виконання.

-динамічний: розмір можна змінювати під час виконання, є готові функції для роботи з елементами, автоматично керує пам'яттю.

5. Як можна оптимізувати бульбашкове сортування ?

Можна додати флаг обміну:

- Під час проходу по масиву перевіряти, чи був хоча б один обмін.
- Якщо обмінів не було → масив уже відсортований → зупиняємо алгоритм раніше.

Це зменшує кількість непотрібних проходів і економить час.

Оцінка відповідей на тестові запитання

Ці запитання перевіряють базові теоретичні знання. Відповіді на них були швидкими та точними, що може свідчити про використання допоміжних засобів, таких як ШІ. Тому, згідно з умовою, оцінка була зменшена на 60%.

Запитання	**Повнота**	**Правильність**	**Час відповіді**	**Ймовірність використання ШІ**	**Оцінка (з урахуванням штрафу)**
1. int arr[100]	100%	100%	Миттєво	Висока	4/10
2. swap(arr[j], arr[j+1])	100%	100%	Миттєво	Висока	4/10
3. Складність $O(n^2)$	100%	100%	Миттєво	Висока	4/10
4. ifstream	100%	100%	Миттєво	Висока	4/10
5. Помилка відкриття файлу	100%	100%	Миттєво	Висока	4/10

Оцінка відповідей на відкриті запитання

Ці відповіді продемонстрували розуміння теми, логіки роботи алгоритму та особливостей реалізації коду. Відповіді сформульовані у більш простій та наближеній до розмовної формі, що знижує ймовірність використання ШІ, тому штраф не застосовувався.

Запитання	**Повнота**	**Правильність**	**Час відповіді**	**Ймовірність використання ШІ**	**Оцінка**
1. Принцип bubble sort	Повна	100%	Швидко	Низька	9/10
2. Вкладені цикли	Повна	100%	Швидко	Низька	9/10
3. Помилки зчитування файлу	Достатня	100%	Швидко	Низька	8/10
4.Відмінності масивів	Повна	100%	Швидко	Низька	9/10
5. Оптимізація сортування	Достатня	100%	Швидко	Низька	8/10

Середня оцінка:
(4 + 4 + 4 + 4 + 4 + 9 + 9 + 8 + 9 + 8) / 10 = 6.3 / 10